Treasury Register -- Phase 2 Implementation Plan

Status: parked. Not started. Picked up when we decide to build the Phase 2 foundation. **Companion to:** `PHASE_2_BLUEPRINT.md` (the what and why) and `.context/roadmap.json` (the ranked backlog). **This document is:** the code-shaped plan a developer executes when the plan is unparked. Package layout, port interfaces, adapter skeletons, migration list, dev stack, sequencing, effort estimates.

Anchor

Ports and adapters. Every external system reaches the app through a port (an interface). Today, each port is bound to a fake or in-house adapter that behaves like the real thing. Day one of a real customer, we bind the port to the real adapter and add credentials. No service code changes.

This gives us: - Every real integration is already tested end-to-end. - The demo VM keeps working forever -- the fakes never leave. - When Anil (or the first paying customer) hands us Oracle admin + Entra tenant + bureau contact, it is a config + credentials job, not a code job.

This decision is consistent with ADR-0004 (`connector-is-first-party`): the Treasury Register Connector is the only place our code speaks Oracle.

---

0. Whole-repo package layout after this plan is executed

```
 backend/
+-- app/
|   +-- connector/                      # NEW -- the whole Phase-2 substrate
|   |   +-- __init__.py
|   |   +-- ports.py                    # every interface
|   |   +-- registry.py                 # binds ports to adapters from env
|   |   +-- retry.py                    # 429/503/500 policy
|   |   +-- health.py                   # writes integration_health
|   |   +-- mapping.py                  # oracle_entity_mapping + gl_account_map lookups
|   |   +-- secrets.py                  # SecretsPort implementations
|   |   +-- adapters/
|   |   |   +-- __init__.py
|   |   |   +-- erp/
|   |   |   |   +-- oracle_fusion.py
|   |   |   |   +-- fake_oracle.py      # binds to seed today
|   |   |   |   \-- csv_dropbox.py      # SFTP + CSV fallback
|   |   |   +-- bank/
|   |   |   |   +-- swift_bureau.py
|   |   |   |   +-- file_bank.py        # dev -- writes pain.001 to disk
|   |   |   |   \-- oracle_cash_mgmt.py # read fallback if customer uses it
|   |   |   +-- market/
|   |   |   |   +-- xignite.py
|   |   |   |   \-- deterministic.py    # today's QuoteEngine, kept forever for tests
|   |   |   +-- identity/
|   |   |   |   +-- oidc.py             # works for Entra / IDCS / Okta / Auth0 / Keycloak
|   |   |   |   \-- local_password.py   # today's flow, kept for `demo` tenants
|   |   |   +-- signing/
|   |   |   |   +-- software.py         # local RSA key, DEV ONLY
|   |   |   |   +-- azure_hsm.py        # Azure Managed HSM
|   |   |   |   \-- pkcs11.py           # customer-owned HSMs
|   |   |   +-- llm/
|   |   |   |   +-- groq.py             # today
|   |   |   |   +-- azure_openai.py
```

```
|   |   |   |   +-- oci_genai.py
|   |   |   |   +-- bedrock.py
|   |   |   |   \-- deterministic.py     # existing fallback logic, moved here
|   |   |   +-- secrets/
|   |   |   |   +-- env_var.py           # today
|   |   |   |   +-- local_file.py        # dev, reads .env.local
|   |   |   |   \-- azure_key_vault.py
|   |   |   \-- mail/
|   |   |       +-- smtp.py              # generic, works with SES/Azure Comms/SendGrid
|   |   |       \-- console.py           # today
|   |   +-- parsers/                     # ISO 20022 + SWIFT MT -- pure functions, no I/O
|   |   |   +-- camt053.py               # bank statement
|   |   |   +-- camt054.py               # credit/debit notification
|   |   |   +-- mt940.py
|   |   |   +-- mt300.py                 # FX confirmation
|   |   |   +-- mt320.py                 # money-market confirmation
|   |   |   +-- pain001.py               # payment initiation
|   |   |   \-- mt202.py
|   |   +-- serialisers/                 # our internal domain -> outbound message
|   |   |   +-- pain001.py
|   |   |   \-- mt202.py
|   |   +-- jobs/                        # scheduled workers
|   |   |   +-- etl_exposure.py          # nightly Oracle AP/AR/EPM pull
|   |   |   +-- etl_bank_statements.py
|   |   |   +-- poster_gl.py             # 15-min GL journal poster
|   |   |   +-- poster_payments.py       # outbound payments
|   |   |   \-- ai_run_reaper.py         # archive ai_run > N months
|   |   \-- cli.py                       # `python -m app.connector <cmd>`
|   +-- services/                        # existing -- untouched except to call ports
|   |   \-- ...
|   +-- api/                             # existing -- untouched
|   \-- ...
+-- tests/
|   +-- connector/
|   |   +-- fixtures/
|   |   |   +-- oracle/                  # respx-recorded JSON responses
|   |   |   +-- bank/                    # sample MT940 / camt.053 / MT300 XML files
|   |   |   \-- xignite/                 # recorded rate responses
|   |   +-- test_ports_contract.py       # every adapter honours its port
|   |   +-- test_oracle_fusion_adapter.py
|   |   +-- test_bank_parsers.py         # parse every sample, round-trip
|   |   +-- test_pain001_serialiser.py   # generates schema-valid XML
|   |   +-- test_oidc_flow.py            # keycloak in docker
|   |   +-- test_market_adapter.py
|   |   \-- test_secrets.py
|   \-- ...
```

---

1. Port interfaces (`app/connector/ports.py`)

The whole substrate hinges on these being right. Each port is a small ABC. Adapters are stateless where possible; state lives in the DB.

```
 # ------------------------------------------------------------ ERP

class ERPPort(Protocol):
   def pull_exposure(self, as_of: date, legal_entity_id: str) -> ExposureSnapshot: ...
   def pull_bank_statement(self, as_of: date, account_id: str) -> BankStatement: ...
```

```
   def post_journal(self, entry: JournalEntry) -> JournalAck: ...

# ------------------------------------------------------------ Bank connectivity

class BankPort(Protocol):
   def submit_payment(self, payment: Payment, signature: bytes) -> BankSubmissionAck: ...
   def poll_inbox(self) -> Iterable[InboundMessage]: ...

# ------------------------------------------------------------ Market data

class MarketDataPort(Protocol):
   def spot(self, pair: str) -> Quote: ...
   def forward(self, pair: str, tenor_months: int) -> Quote: ...
   def benchmark(self, name: str, as_of: date) -> Rate: ...

# ------------------------------------------------------------ Identity

class IdentityPort(Protocol):
   def start_login(self, redirect_uri: str) -> LoginStart: ...
   def complete_login(self, code: str, state: str) -> IdentityAssertion: ...
   def verify_bearer(self, token: str) -> IdentityAssertion | None: ...

# ------------------------------------------------------------ Signing

class SigningPort(Protocol):
   def sign(self, payload: bytes, key_ref: str) -> bytes: ...

# ------------------------------------------------------------ LLM

class LLMPort(Protocol):
   def chat(self, system: str, user: str, model_hint: str | None = None) -> LLMResponse: ...

# ------------------------------------------------------------ Secrets

class SecretsPort(Protocol):
   def get(self, name: str) -> str: ...
   def rotate(self, name: str) -> None: ...  # no-op in most adapters

# ------------------------------------------------------------ Mail

class MailPort(Protocol):
   def send(self, to: str, subject: str, body_text: str, body_html: str | None = None) -> None: ...
```

Every service that today calls `os.environ.get(...)`, `requests.post(...)`, `httpx.get(...)`, `groq_client.chat...` or reads a seed row moves to `port.<method>(...)` via a dependency-injected registry.

---

2. Adapters -- real + fake, per port

Every port ships **at least one adapter that runs today with zero external dependency** and **at least one real adapter that is contract-tested against fixtures**. The dev never has to leave the laptop to build.

Port	Ships day-1 (dev / demo)	Real Phase-2 adapter	Contract test uses
ERPPort	`FakeOracle` (reads seed)	`OracleFusion`	`respx` fixtures recorded from Oracle OpenAPI descriptor

Port	Ships day-1 (dev / demo)	Real Phase-2 adapter	Contract test uses
BankPort	`FileBank` (writes pain.001 to `dev/outbox/`)	`SwiftBureau` (SFTP)	Sample MT/camt files from public ISO 20022 catalogue
MarketDataPort	`Deterministic` (today's `QuoteEngine`)	`Xignite`	Recorded HTTP fixtures
IdentityPort	`Keycloak` (local docker)	`OIDC` (works with Entra, IDCS, Okta, Auth0)	Keycloak in docker-compose
SigningPort	`Software` (RSA key in vault, DEV ONLY)	`AzureHSM`, `PKCS11`	Software adapter tests + interface contract
LLMPort	`Groq` (today)	`AzureOpenAI`, `OciGenAI`, Bedrock	Recorded chat responses + `Deterministic` fallback
SecretsPort	`EnvVar` / `LocalFile`	`AzureKeyVault`	Azurite / mocked SDK
MailPort	`Console` (today) + `Smtp -> Mailhog` (docker)	`Smtp -> SES/AzureComms/SendGrid`	Mailhog captures + asserts

Rule: a fake adapter that returns hard-coded happy-path data is not enough. Every fake also has an `on_error` mode toggled by env var so we can assert the whole error path (retry, exception surfacing, oncall alert) in tests without a real dependency.

---

3. New tables (all additive, all nullable, no existing shape touched)

One migration per table. All safe to apply now, populated only when the corresponding adapter lands.

Table	Purpose	Feeds
`oracle_entity_mapping`	`BusinessUnit` (Oracle) -> `legal_entity_id` (us), per customer	ERP inbound
`gl_account_map`	(event_kind, legal_entity_id) -> Oracle `AccountCombination`	ERP outbound
`sso_identity`	IdP subject (`sub`) -> `app_user.id`	Identity
`sso_group_map`	IdP group claim -> our `membership.role`	Identity
`ai_run`	Every LLM call -- closes Q-004	LLM + compliance page
`cash_balance`	Daily bank balances per account per currency	Bank / cash management
`market_snapshot`	Persisted rates USED by a write (not just seen)	Market data

Table	Purpose	Feeds
`bank_ack`	Outbound payment acknowledgements (UETR, status, reason code)	Bank outbound
`gl_posting_ack`	GL journal acks (`JeHeaderId` from Oracle)	ERP outbound
`integration_health`	Last-successful-run per adapter + last-error	Admin observability
`data_snapshot`	Point-in-time snapshots for compliance replay (P1-06)	Compliance

Order matters only in one place: `ai_run` is safe to ship immediately (no dependency); everything else waits for its adapter.

---

4. Dev stack -- `docker-compose up` gives a working system

Extension of the existing `docker-compose.yml`:

```
services:
 postgres:      # already there
 keycloak:      # OIDC dev -- realm preloaded with a Treasury Register client + 5 test users
 redis:         # rate-limit multi-instance target + job queue
 mailhog:       # SMTP catcher -- invite emails are visible in a web UI
 wiremock:      # optional -- pretends to be Oracle Fusion Cloud; loaded with recorded scenarios
```

Every dev on the team runs `docker compose up -d`, sets `TREASURY_DATABASE_URL=postgres://...`, `OIDC_ISSUER_URL=http://localhost:8080/realms/treasury-dev`, and has SSO + Postgres + queue + email + mock-Oracle working locally in five minutes.

---

5. Sequencing -- what to build in what order

Ranked by what unblocks the most. Each stage has an explicit **Definition of Done**.

Stage 1 -- Ports skeleton + fakes for what we already do (~1 day)

Build `app/connector/ports.py`, `registry.py` and adapters that reproduce today's behaviour behind the port. Nothing user-visible changes.

- `FakeOracle` reads `seed_data.py` and returns `ExposureSnapshot` rows shaped exactly like `currency_exposure`.
- `Deterministic` market adapter wraps the existing `QuoteEngine`.
- `LocalPassword` identity adapter wraps today's `IdentityService`.
- `EnvVar` secrets adapter wraps today's `os.environ.get`.
- `Console` mail adapter wraps today's `print("email would go to ...")`.
- Every service that used to call a thing directly, now calls a port. Behaviour unchanged. 289 tests still green.

DoD: `pytest` green; `grep os.environ backend/app/services` returns zero for anything a customer would rotate.

Stage 2 -- Every new empty table (~half day)

Ship the 11 migrations from §3. Additive, nullable, no data touched. Test suite proves migrations replay cleanly on both SQLite + Postgres (Postgres CI job already exists).

DoD: `alembic upgrade head` green on both dialects; new tables exist but are empty; existing 289 tests unchanged.

Stage 3 -- `ai_run` end-to-end + Compliance AI Activity tab (~1 day)

No external dependency. Wire every Groq call through `LLMPort` (already skeletonised in Stage 1); write `ai_run` on every call; add the fifth Compliance tab that reads it.

DoD: Every LLM call in the app writes an `ai_run` row; compliance page shows AI activity; Q-004 closed.

Stage 4 -- OIDC via local Keycloak (~2–3 days)

The keystone of every "we can hand this to a customer" story. All IdP-agnostic.

- Keycloak in docker-compose with a preloaded realm.
- OIDC adapter (Authlib).
- `sso_identity` + `sso_group_map` populated on first sign-in.
- Local-password sign-in kept for `demo` tenants (feature flag on `tenant`).
- Test: `demo` user signs in with password; `real` user signs in via Keycloak; both land at the same audit event.

DoD: A dev logs in through Keycloak, lands on the treasurer page, has the right role, and their sign-in writes an `audit_event(action=user.signed_in, actor_source=oidc)`.

Stage 5 -- Bank message parsers + serialisers (~2–3 days)

Real ISO 20022 XSDs + SWIFT MT format. No bank required.

- `camt053`, `camt054`, `mt940`, `mt300`, `mt320`, `pain001`, `mt202` -- parsers and, for outbound formats, serialisers.
- Sample messages in the repo at `tests/connector/fixtures/bank/` (public samples, redistributable).
- Test: every sample parses; every serialised pain.001 validates against the pain.001 XSD.
- `FileBank` adapter: outbound writes to `dev/outbox/`; inbound reads from `dev/inbox/`.

DoD: A developer drops a real bank sample into `dev/inbox/`, hits an endpoint, and sees the statement in our `cash_balance` table. Same code will read from SFTP when we bind `SwiftBureau`.

Stage 6 -- Market data with recorded fixtures (~1 day)

- `XigniteAdapter` real HTTP shape.
- `respx` fixtures for `GetRealTimeRate`, `GetForwardRate`, error responses.
- Runtime binds `Deterministic` today; a config flip binds `Xignite`.
- Every hedge/deal write persists the rate used -- `market_snapshot` gets populated.

DoD: Set `MARKET_DATA_PROVIDER=xignite` and (with a valid API key in the vault) the app hits the real endpoint; tests still green with `deterministic`.

Stage 7 -- Oracle Fusion Cloud REST client + `respx` fixtures (~3–4 days)

The big one, still without needing an Oracle person.

- Real HTTP client for `invoices`, `receivablesInvoices`, `journalEntries`, `bankStatements` -- pagination, `hasMore` / `offset`, `ExternalReference` idempotency, error envelope handling.
- IDCS/Identity Domains OAuth2 client-credentials token flow.

- `respx` fixtures recorded from Oracle's public OpenAPI descriptor; if we sign up for the free Oracle Cloud Applications trial we record from a real pod for the tricky endpoints.
- `oracle_entity_mapping` and `gl_account_map` admin surface (small CLI + one admin page).
- Contract tests for every retry policy row from `PHASE_2_BLUEPRINT.md §3.3`.

DoD: `pytest tests/connector/test_oracle_fusion_adapter.py` covers every read + write shape against fixtures; a customer's Oracle admin can hand us IDCS creds and we plug them into the vault, flip `ERP_PROVIDER=oracle_fusion`, and shadow-mode runs cleanly.

Stage 8 -- Signing + secrets vault production adapters (~2 days)

- `AzureKeyVaultAdapter` for secrets (SDK + tests against Azurite or mocked SDK).
- `SoftwareSigningAdapter` today (RSA key from vault) -- pen-tested internally, DEV ONLY marker.
- `AzureManagedHSMAdapter` skeleton, contract-tested against the software adapter's outputs.
- `PKCS11Adapter` skeleton for customer HSMs.

DoD: Runtime binds `Software` today; flipping `SIGNING_PROVIDER=azure_hsm` (with the right config) is expected to work without code change.

Stage 9 -- Structured logs + Sentry wiring + integration_health admin page (~2 days)

- `structlog` configured; every service adds correlation IDs (deal_id, actor_user_id, tenant_id).
- Sentry SDK installed. `SENTRY_DSN=""` = no-op.
- Every adapter writes `integration_health` rows on success + failure.
- New admin page: table of every integration with last-success / last-error / retry count. Zero external dependency to render.

DoD: Admin page tells you "Oracle: last success 3h ago; Xignite: DOWN since 09:12 with error 'API key invalid'". The customer can see it too.

Stage 10 -- Job queue abstraction + workers (~2 days)

The ETL, GL poster and payment poster all need a scheduler.

- `Job` abstraction. Poll-the-table implementation using our existing DB (Postgres skip-locked, SQLite queue table).
- Optional Redis backend for multi-instance.
- Runs in-process during dev; workers as a separate process in prod (`python -m app.connector.jobs run --worker`).
- Every job writes to `integration_health` and emits structured logs.

DoD: ETL, GL poster, payment poster all run as jobs. Same code in-process (dev) or separate worker (prod).

---

6. Rough effort total

~2 weeks of solo dev, or ~1 week if two people split parallel stages (1–3 -> Dev A, 4–5 -> Dev B).

Every stage is independently shippable. Nothing in this plan requires an Azure person, an Oracle person or a bank contact. When those people arrive, they hand us credentials -- we hand back a working integration.

---

7. Config knobs introduced

Every one has a safe default matching today's behaviour. Nothing in this plan changes the running app until a knob is flipped.

Env var	Default	Options
`ERP_PROVIDER`	`fake_oracle`	`oracle_fusion`, `csv_dropbox`
`BANK_PROVIDER`	`file_bank`	`swift_bureau`, `oracle_cash_mgmt`
`MARKET_DATA_PROVIDER`	`deterministic`	`xignite`, `refinitiv`, `bloomberg`
`IDENTITY_PROVIDER`	`local_password`	`oidc` (with `OIDC_ISSUER_URL` set)
`SIGNING_PROVIDER`	`software`	`azure_hsm`, `pkcs11`
`LLM_PROVIDER`	`groq`	`azure_openai`, `oci_genai`, `bedrock`, `dete`
`SECRETS_PROVIDER`	`env_var`	`local_file`, `azure_key_vault`
`MAIL_PROVIDER`	`console`	`smtp` (with `SMTP_HOST` etc.)
`JOB_STORE`	`db`	`redis`
`OIDC_ISSUER_URL`	unset	e.g. `https://login.microsoftonline.com/<ter`
`SENTRY_DSN`	unset	Sentry DSN string
`TREASURY_DATABASE_URL`	`sqlite:///backend/treasury.db`	`postgresql+psycopg://...` (P0-01, shipped)

---

8. Test strategy

- **Unit tests** already cover services + repos. Unchanged.
- **Contract tests** -- every port has one file that runs every adapter through the same suite. Guarantees interchangeability.
- **Adapter integration tests** -- each real adapter runs against fixtures (respx for Oracle + Xignite, sample XML/MT files for bank, keycloak for OIDC). Nothing hits a live external endpoint in CI.
- **End-to-end smoke** -- `docker compose up`, then `pytest tests/e2e/` walks: sign in via Keycloak -> propose a deal -> GL journal serialised -> payment file written to `dev/outbox/`. Confirms every port wires end-to-end with no real vendor involved.

---

9. Explicitly NOT in this plan

- **Data migration from SQLite to Postgres** -- see ADR-0018. Independent of adapters; handled per customer.
- **Choosing between OCI Gen AI vs Azure OpenAI** -- depends on customer residency; both adapters ship, choice is config.
- **Onboarding a specific customer's Oracle pod** -- that's an implementation project after this substrate lands.
- **Compliance/SOC 2/ISO 27001** -- separate track; benefits from this substrate but isn't gated by it.

- **Multi-tenancy strategy (Q-002)** -- orthogonal.
- **Any actual UI change beyond the Compliance AI Activity tab (Stage 3) and Integration Health admin page (Stage 9)** -- every other change is behind the scenes.

---

10. How this plan is unparked

When we decide to build, the developer:

1. Opens this document + `PHASE_2_BLUEPRINT.md` (business layer) + `.context/roadmap.json` (priorities).
2. Picks Stage 1. Ships it.
3. Repeats.

Each stage's Definition of Done is explicit. No stage takes more than a week solo. The developer can pause between any two stages and the app keeps working -- because nothing in this plan changes runtime behaviour without a config flip.

---

11. What we'll have at the end

A product where every real-world integration is already coded, tested and running against a fake or dev-only backend. When Anil (or any first customer) says "here are our systems", we run:

```
 # Fill in a .env for their tenant
ERP_PROVIDER=oracle_fusion
IDENTITY_PROVIDER=oidc
OIDC_ISSUER_URL=https://login.microsoftonline.com/<their-tenant>/v2.0
BANK_PROVIDER=swift_bureau
MARKET_DATA_PROVIDER=xignite
SIGNING_PROVIDER=azure_hsm
LLM_PROVIDER=azure_openai
SECRETS_PROVIDER=azure_key_vault
MAIL_PROVIDER=smtp
# ...
```

Restart. Everything works because everything was already working -- against Keycloak, Mailhog, Xignite fixtures, ISO 20022 samples and an Oracle-shaped respx mock. Only the endpoints changed.